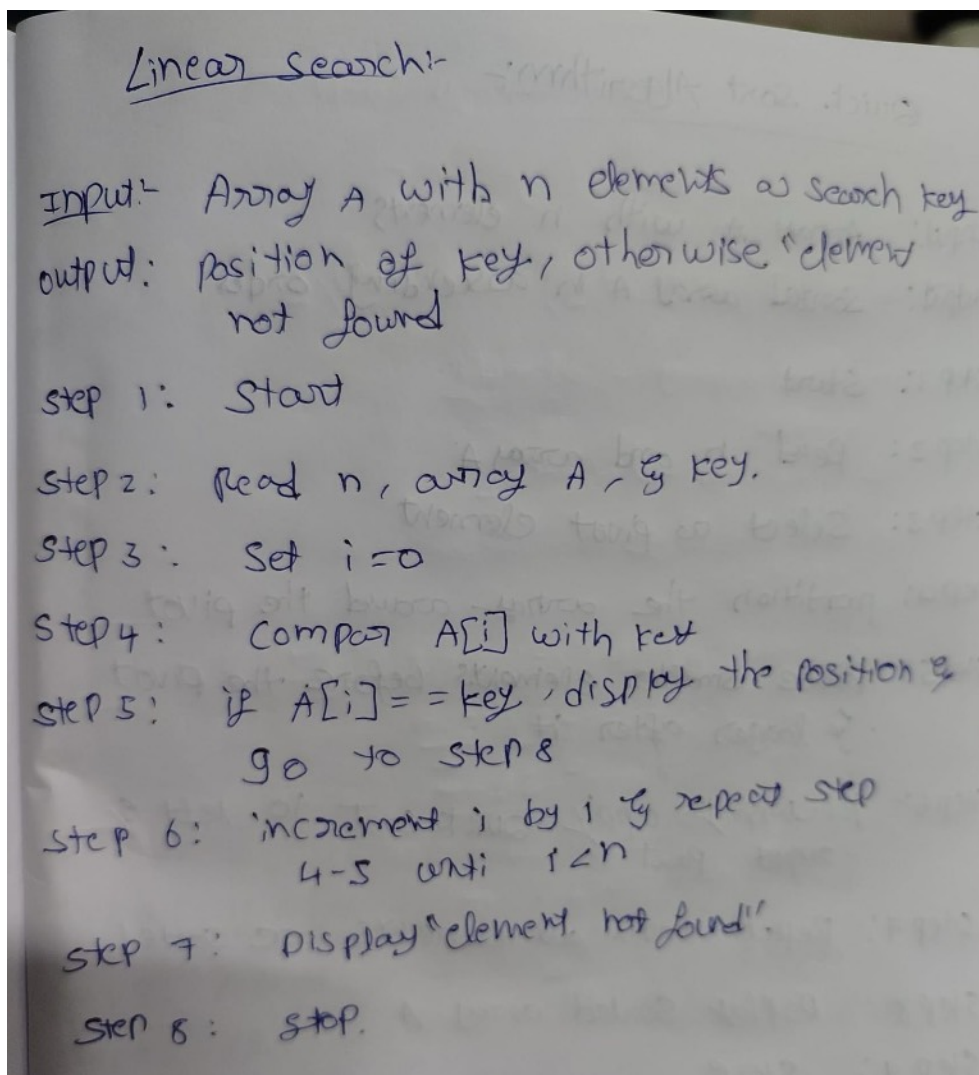


PRACTICAL-2

AIM: To implement and analyze the time complexity of Linear Search and Binary Search algorithms, and compare their searching efficiency for different input sizes.

LINEAR SEARCH:

ALGORITHM:



Linear Search:-

Input:- Array A with n elements & search key
output: position of key, otherwise "element not found"

step 1: Start

step 2: Read n, array A, & key.

step 3: Set $i = 0$

step 4: Compare $A[i]$ with key

step 5: if $A[i] == \text{key}$, display the position & go to step 8

step 6: increment i by 1 & repeat step 4-5 until $i < n$

step 7: Display "element not found".

step 8: stop.

PROGRAM/CODE:

```
#include <iostream>
using namespace std;

int main()
{
    int size, arr[100], key, i;

    cout << "Enter the size of the array: ";
    cin >> size;

    cout << "Enter " << size << " elements: ";
    for (i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    cout << "Enter the element to be searched: ";
    cin >> key;

    i = 0;

    for (i = 0; i < size; i++)
    {
        if (arr[i] == key)
        {
            cout << "Element found at position " << i + 1;
            break;
        }
    }

    if (i == size)
    {
        cout << "Element not found";
    }

    return 0;
}
```

OUTPUT:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

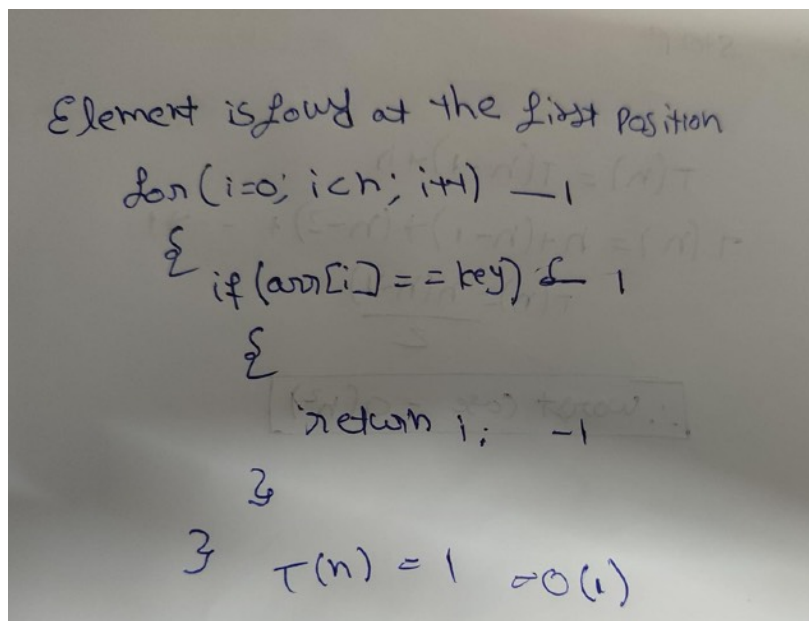
nandhagopal@NGKs-MacBook-Air c++ % /Users/nandhagopal/Documents/CODING/c++/linearsearch
Enter the size of the array: 4
Enter 4 elements: 23
55
11
88
Enter the element to be searched: 23
Element found at position 1
❖ nandhagopal@NGKs-MacBook-Air c++ %

```

TIME COMPLEXITY:

1.BEST CASE:

The best case occurs when the search element is found at the first position.



2.AVERAGE CASE:

The average case occurs when the element is found somewhere in the middle of the array.

3.WORST CASE:

The worst case occurs when the element is at the last position or is not present in the array.

Time Complexity for Linear Search:-

```

for(i=0; i<size; i++) → 1 + (n+1) + n = 2n+2
{
    if(arr[i] == key) → n
    {
        cout << "Element found at position" << i+1; → 1
        break; → 1
    }
}
if (i == size) → 1
{
    cout << "Element not found"; → 1
}

```

$T(n) = 2n + 2 + n + 1 + 1 + 1$
 $= 3n + 5$
 $\therefore T(n) = O(n)$

Best case: $O(1)$
Average case: $O(n)$
Worst case: $O(n)$

BINARY SEARCH:

ALGORITHM:

Binary Search Algorithm:-

input:- Sorted array A with n element & a search element

output:- Position of key if found, otherwise "element not found"

step 1: Start

step 2: Read n Sorted array A & key

step 3: set low = 0 & high = n-1

step 4: find $mid = (low + high) / 2$

step 5: if $A[mid] == key$, display the position

step 6: if $A[mid] < key$, set $low = mid + 1$

step 7: if $A[mid] > key$, set $high = mid - 1$

step 8: Repeat 4-7 while $low < high$

step 9: If the element is not found, display "element not found"

step 10: Stop

PROGRAM/CODE:

```
#include <iostream>
using namespace std;

int BinarySearch(int arr[], int size, int search)
{
    int low = 0;
    int high = size - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (search == arr[mid])
        {
            return mid;
        }
    }
}
```

```
}
else if (search < arr[mid])
{
    high = mid - 1;
}
else
{
    low = mid + 1;
}
}

return -1;
}

int main()
{
    int size, arr[100], search, result;

    cout << "Enter the size of the array: ";
    cin >> size;

    cout << "Enter " << size << " elements in sorted order: ";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    cout << "Enter the element to be searched: ";
    cin >> search;

    result = BinarySearch(arr, size, search);

    if (result != -1)
    {
        cout << "Element found at position " << result + 1;
    }
    else
    {
        cout << "Element not found";
    }

    return 0;
}
```

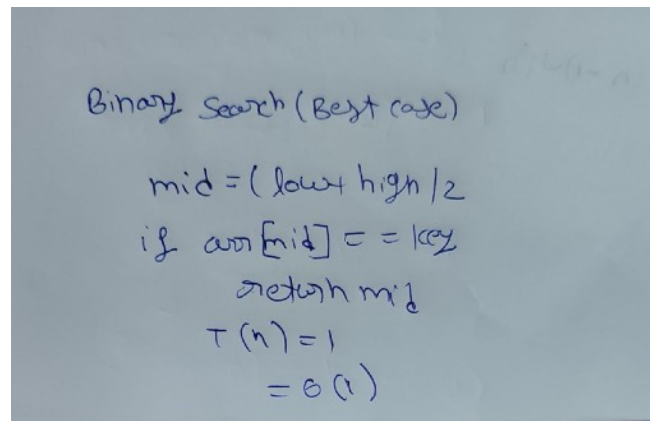
OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• nandhagopal@NGKs-MacBook-Air c++ % /Users/nandhagopal/Documents/CODING/c++/binarysearch
Enter the size of the array: 4
Enter 4 elements in sorted order: 22
11
44
77
Enter the element to be searched: 44
Element found at position 3%
❖ nandhagopal@NGKs-MacBook-Air c++ %
```

TIME COMPLEXITY:

1.BEST CASE:

The best case occurs when the search element is found at the middle position in the first comparison.



2.AVERAGE CASE:

The average case occurs when the element is found after several divisions of the search space.

3.WORST CASE:

The worst case occurs when the element is found after maximum divisions or is not present in the array.

Time complexity for Binary Search:-

```

int BinarySearch(arr, size, search)
{
    low = 0
    high = size - 1;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (search == arr[mid])
        {
            return mid;
        }
        else if (search < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }
    return -1;
}

```

$\rightarrow 1$
 $\rightarrow 1$
 $\rightarrow \log_2 n + 1$
 $\rightarrow \log_2 n$
 $\rightarrow \log_2 n$
 $\rightarrow 0$
 $\rightarrow \log_2 n$
 $\rightarrow \log_2 n$
 $\rightarrow \log_2 n$
 $\rightarrow 1$

$T(n) = 1 + 1 + (k+1) + k + k + k + k + 1$
 $T(n) = 5k + 4$
 $k = \log_2 n$
 $T(n) = 5 \log_2 n + 4$
 $T(n) = O(\log n)$

Best case:- $O(1)$ worst case:- $O(\log n)$
Average case:- $O(\log n)$ space:- $O(1)$

CONCLUSION:

Thus, Linear Search and Binary Search were successfully implemented and their time complexities were analyzed. It was observed that Linear Search has a time complexity of $O(n)$, while Binary Search has $O(\log n)$, making Binary Search more efficient for searching in a sorted array.